

Improving Patient Outcomes through Predictive Analytics: Predicting ICU Readmission Using NLP and AI Models.

Michigan Technological University

Andrea Wroblewski

Group 1

April 2024



Goal

The goal of this project is to develop and evaluate a predictive analytics model utilizing machine learning and natural language processing (NLP) techniques to accurately identify patients at risk of ICU readmission within 30 days.



Background

- ICU readmissions occur in approximately 16% of cases within 30 days post-discharge, with significantly higher mortality rates (21–40%) compared to non-readmitted patients (3.6–8.4%). (Sheetrit, E. et al., 2023).
- These readmissions increase healthcare costs, strain resources, and highlight the need for early detection strategies (Rojas et al., 2018).



Literature Review



Study / Model	Type	Key Findings
GBM (Rojas et al., 2018)	Structured Data	AUROC 0.76 using EHR features. Solid performance with traditional ML.
Discharge Notes (Sheetrit et al., 2023)	Unstructured Text	Text-based notes outperformed structured data, showing richer clinical signals.
Text-Based NLP Model (Orangi-Fard et al., 2022)	Unstructured Text	AUROC improved from 0.71 to 0.74 after feature selection (825 words) using SVM-RBF
BERTopic + LSTM (Chiu et al., 2024)	Unstructured Text	Leveraged topic modeling + LSTM to extract semantic patterns from notes.
Optimized XGBoost (González-Nóvoa et al., 2023)	Structured Data	Achieved AUROC 0.92, showing benefits of hyperparameter tuning.
ClinicalBERT (Alsentzer et al., 2019)	Unstructured Text (MIMIC)	Outperformed BioBERT and general BERT by capturing domain-specific context.
MedText (Lu et al., 2021)	Hybrid (Text + Knowledge Graph)	AUROC 0.825 by combining discharge summaries with UMLS knowledge graphs.

Data Sets



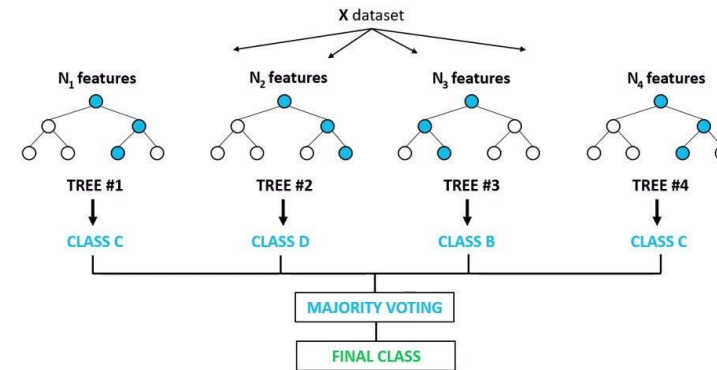
- **MIMIC-IV**
 - Version: 2.2
 - <https://physionet.org/content/mimiciv/2.2/>
- **MIMIC-IV-Note: Deidentified free-text clinical notes**
 - Version: 2.2
 - <https://physionet.org/content/mimic-iv-note/2.2/>



Models Used

- Random Forest Classifier
- SMOTE
 - Synthetic Minority Over-sampling Technique
- XGBoost Classifier
 - Extreme Gradient Boosting
- ClinicalBERT
 - Bidirectional Encoder Representations from Transformers

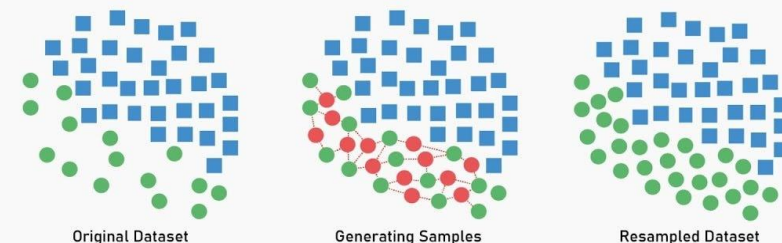
Random Forest Classifier



SMOTE

HANDLE IMBALANCED DATASET

Synthetic Minority Oversampling Technique



Code: Libraries Used

Data manipulation and OS interaction

import numpy as np # Numerical operations

import pandas as pd # Data handling

import os # Operating system interactions

Visualization

import matplotlib.pyplot as plt # Plotting graphs

import seaborn as sns

from sklearn.metrics import (

 classification_report, # Precision, recall, f1-score

 roc_auc_score, # AUC for ROC

 roc_curve,

 confusion_matrix,

 ConfusionMatrixDisplay # Display confusion matrix

)

Machine learning models and tuning

from sklearn.ensemble import RandomForestClassifier # Random Forest model

from xgboost import XGBClassifier # XGBoost classifier

from sklearn.model_selection import train_test_split, RandomizedSearchCV # Splitting & tuning

NLP & deep learning (for ClinicalBERT embeddings later)

import torch

from transformers import AutoTokenizer, AutoModel

from tqdm import tqdm # Progress bar

Handling class imbalance

from imblearn.over_sampling import SMOTE # Synthetic Minority Over-sampling Technique



Code: Structured Data

```
patients_path = "S:/Downloads/mimic-iv-2.2/mimic-iv-2.2/hosp/patients.csv.gz" #Patient demographic Info
icustays_path = "S:/Downloads/mimic-iv-2.2/mimic-iv-2.2/icu/icustays.csv.gz" #ICU admission and discharge record
diagnoses_path = "S:/Downloads/mimic-iv-2.2/mimic-iv-2.2/hosp/diagnoses_icd.csv.gz" #ICD diagnosis code

df_patients = pd.read_csv(patients_path)
df_icustays = pd.read_csv(icustays_path)

diagnosis_chunks = [] #Temporary list to hold filtered chunks
target_subject_ids = set(df_patients['subject_id'].unique()) #Keep only relevant patient

#Process diagnoses.csv in chunks of 50,000 rows
for chunk in pd.read_csv(diagnoses_path, chunksize=50000):
    filtered_chunk = chunk[chunk['subject_id'].isin(target_subject_ids)]
    diagnosis_chunks.append(filtered_chunk) #Append each filtered chunk

#Combine all chunks into one DataFrame
df_diagnoses = pd.concat(diagnosis_chunks, ignore_index=True)
```



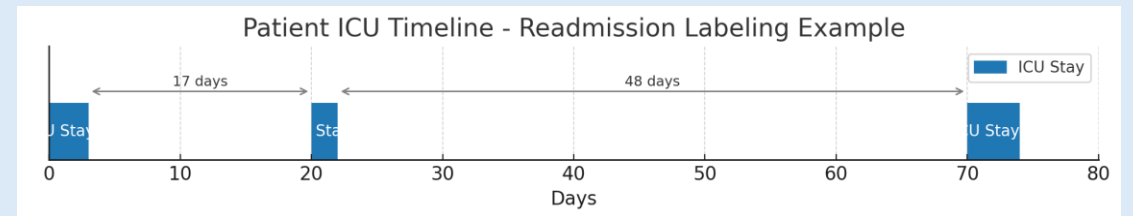
Code: Structured Data

```
df_icustays['intime'] = pd.to_datetime(df_icustays['intime'])
df_icustays['outtime'] = pd.to_datetime(df_icustays['outtime'])
df_icustays = df_icustays.sort_values(by=['subject_id', 'intime']).reset_index(drop=True)
```

```
readmission_labels = []
"""Empty list to hold readmission data
0 = Not Readmitted
1= Readmitted to ICU within 30 days
"""

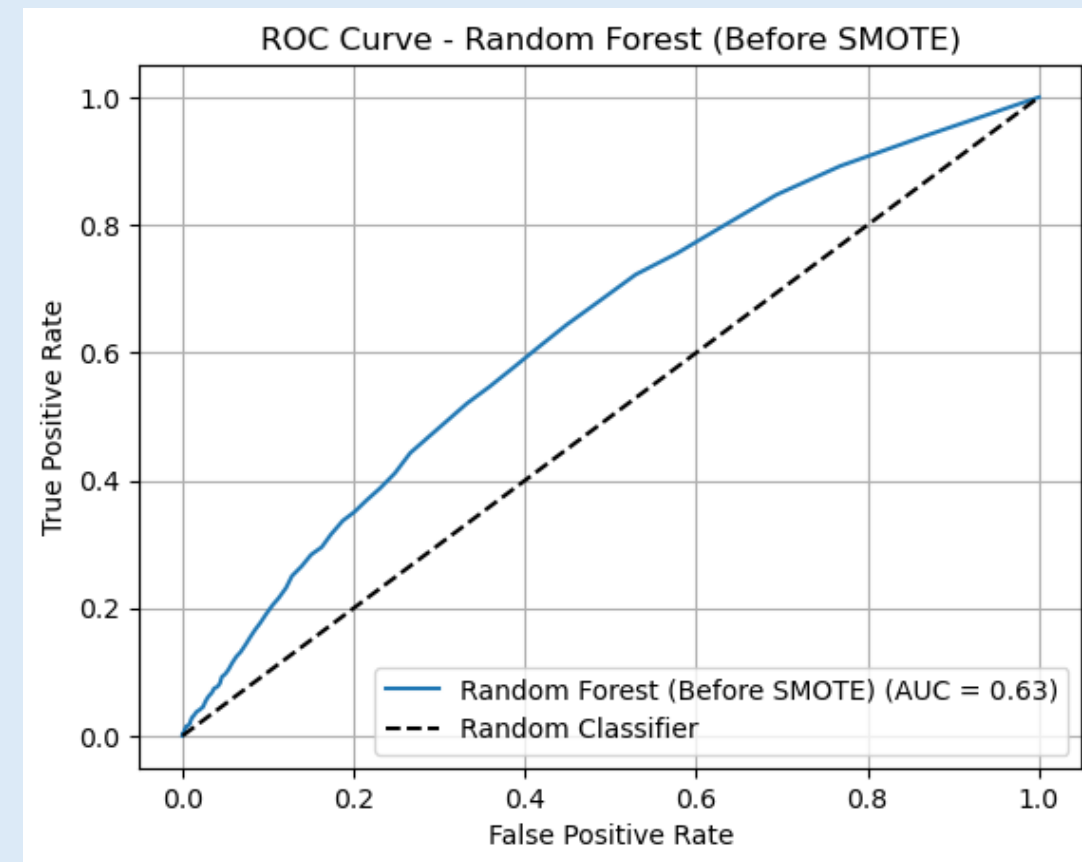
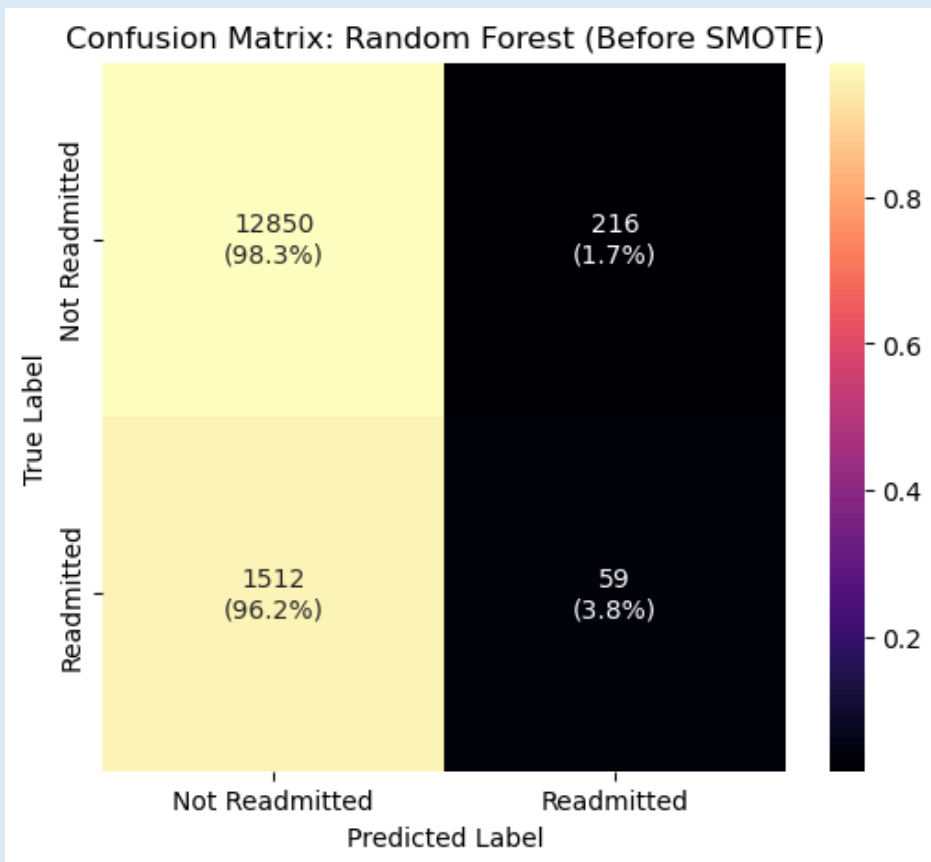
# Loop through ICU stays grouped by patient
for subject_id, stays in df_icustays.groupby('subject_id'): #A dataframe that has patient ICU stays
    stays = stays.sort_values(by='intime').reset_index(drop=True)
    readmitted = [0] * len(stays) #List of 0 and 1 for ICU readmission
    for i in range(len(stays) - 1):
        delta = (stays.loc[i + 1, 'intime'] - stays.loc[i, 'outtime']).days #Calculates number of days between discharge and the
next admission
        if 0 < delta <= 30: #If patient was not readmitted to the ICU in 30 Days
            readmitted[i] = 1 #If readmitted 1
        readmission_labels.extend(readmitted) #Adds readmitted data to list

df_icustays = df_icustays.iloc[:len(readmission_labels)] #Data frame only has as many rows as labels
df_icustays['readmitted'] = readmission_labels #Adds readmitted column to ICU stays data set
```





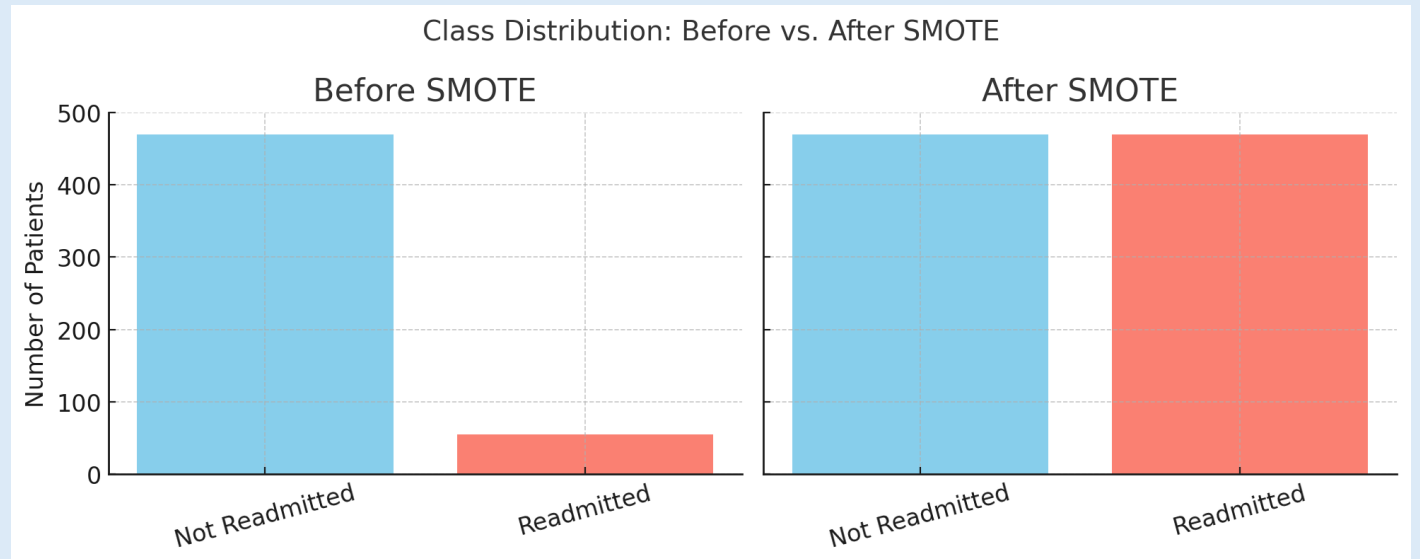
Structed Data: Random Forest Pre-SMOTE



Code: Structured Data

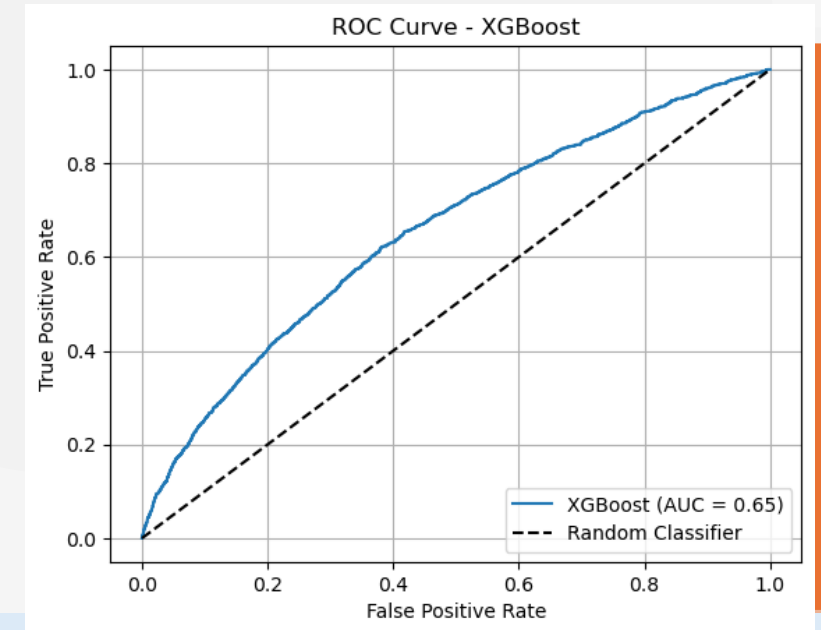
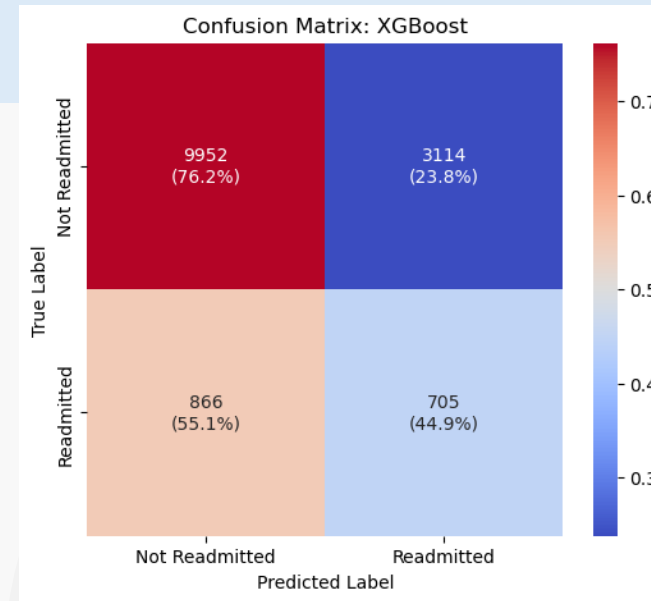
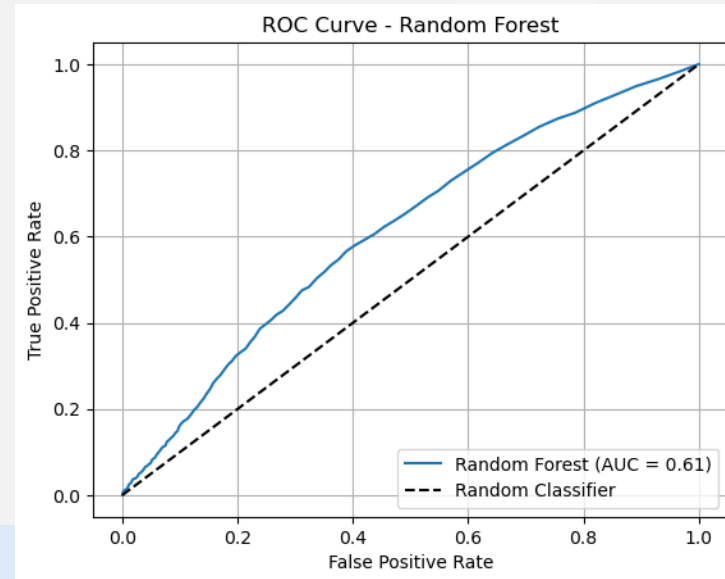
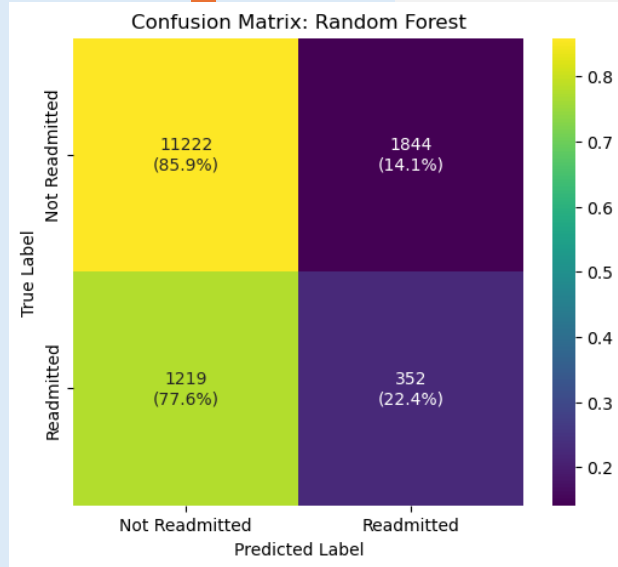
```
#Apply SMOTE to Balance Classes  
smote = SMOTE(random_state=42)
```

```
# Apply SMOTE only on training data to prevent data leakage  
X_resampled, y_resampled = smote.fit_resample(X_train, y_train)
```

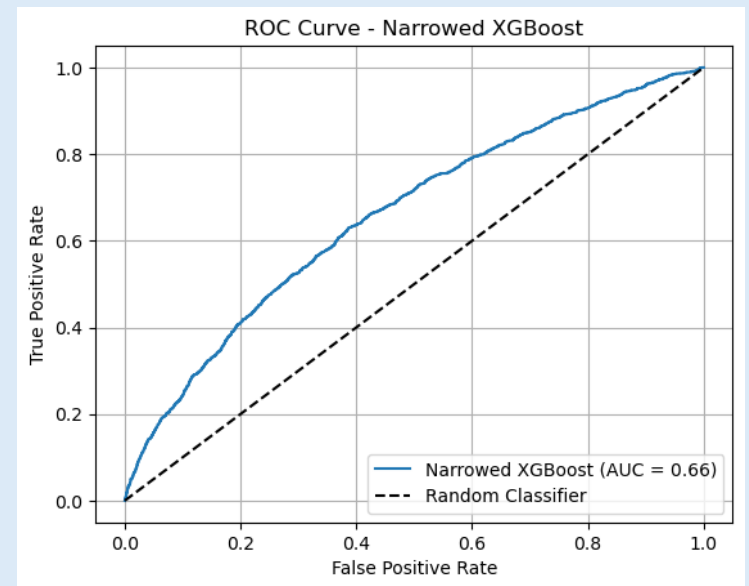
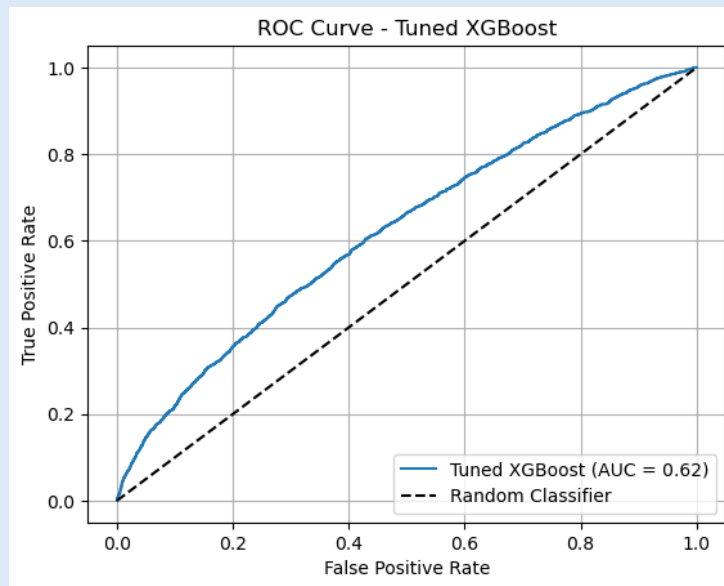
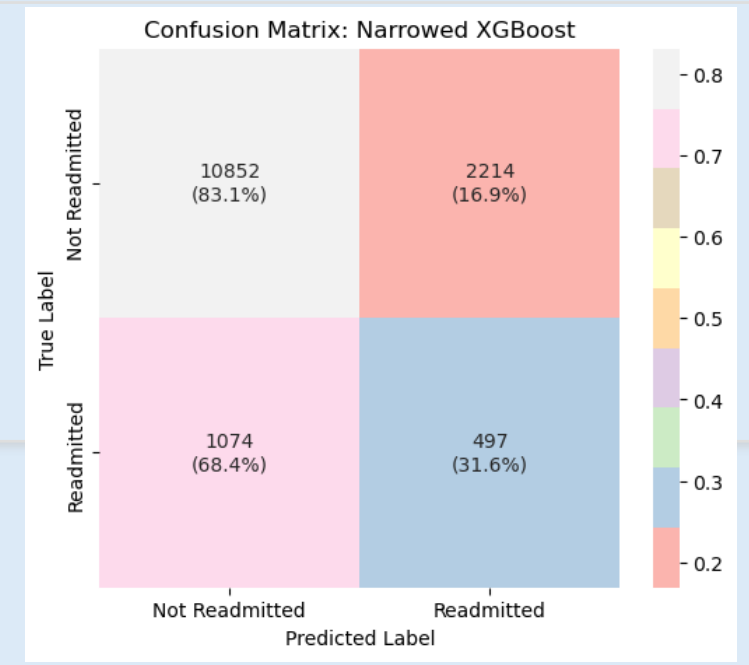
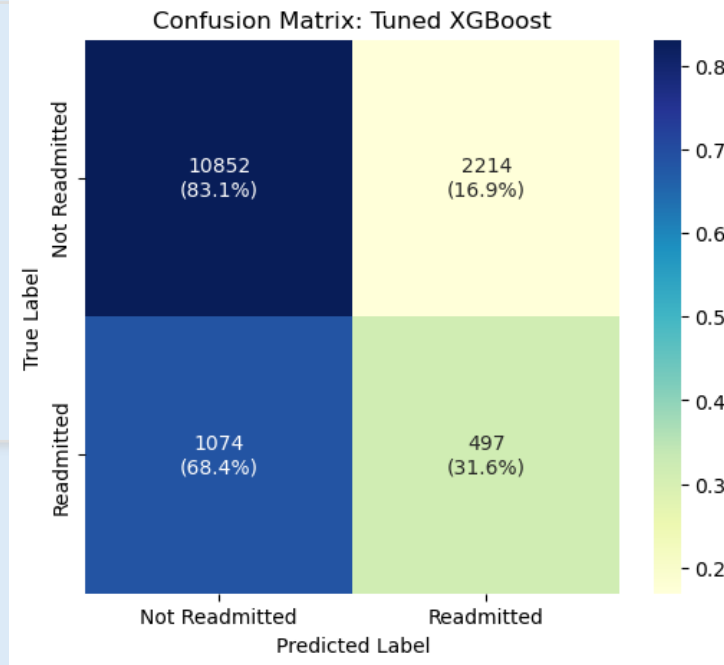


Results:

Structed Data after SMOTE



Results: Structed Data



Code: Unstructured Data

```
#Load the compressed discharge notes file
df_raw = pd.read_csv(
    r"S:\Downloads\mimic-iv-note-deidentified-free-text-clinical-notes-2.2\note\discharge.csv.gz",
    compression='gzip'
)

# Preview the data
print("Raw rows loaded:", len(df_raw))
print("Unique note types:", df_raw['note_type'].unique())
print(df_raw.head())

# Filter for Discharge Summaries
df_notes = df_raw[df_raw['note_type'] == 'DS']

# Keep only necessary columns
df_notes = df_notes[['hadm_id', 'text']].dropna()
df_notes = df_notes.drop_duplicates(subset='hadm_id')
df_notes = df_notes[df_notes['text'].str.strip() != ""].reset_index(drop=True)

#Check
print("Cleaned discharge summaries:", len(df_notes))
print(df_notes.head())
```



Code: Unstructured Data

```
tokenizer = AutoTokenizer.from_pretrained("emilyalsentzer/Bio_ClinicalBERT")
model = AutoModel.from_pretrained("emilyalsentzer/Bio_ClinicalBERT")
```

```
# Define embedding function
```

```
def embed_text_bert(texts, tokenizer, model, device='cpu', max_length=512):
    embeddings = []
    model.to(device)
    model.eval()
```

```
    with torch.no_grad():
```

```
        for text in tqdm(texts, desc="Embedding discharge summaries"):
```

```
            inputs = tokenizer(
                text,
                return_tensors="pt",
                truncation=True,
                padding="max_length",
                max_length=max_length
            )
```

```
            inputs = {key: val.to(device) for key, val in inputs.items()}
```

```
            outputs = model(**inputs)
```

```
            cls_embedding = outputs.last_hidden_state[:, 0, :] # Take CLS token
```

```
            embeddings.append(cls_embedding.cpu().numpy().squeeze())
```

```
    return np.vstack(embeddings)
```

```
texts = df_notes['text'].iloc[:2500].tolist() # Discharge summaries
```

```
hadm_ids = df_notes['hadm_id'].iloc[:2500].tolist() # Corresponding hospital admission IDs
```

```
# Generate BERT Embeddings
```

```
bert_array = embed_text_bert(texts, tokenizer, model)
```

```
# Create DataFrame for embeddings
```

```
df_bert_embeddings = pd.DataFrame(
    bert_array,
    columns=[f"bert_feature_{i}" for i in range(bert_array.shape[1])]
)
```

```
# Attach hospital admission ID for merging with other clinical data later
```

```
df_bert_embeddings['hadm_id'] = hadm_ids
```

```
# Save embeddings for reuse
```

```
df_bert_embeddings.to_pickle("bert_embeddings.pkl")
```

```
print("BERT embeddings saved as bert_embeddings.pkl")
```

```
# Preview
```

```
print("ClinicalBERT embeddings shape:", df_bert_embeddings.shape)
```

```
print(df_bert_embeddings.head())
```



Code: Combining Datasets

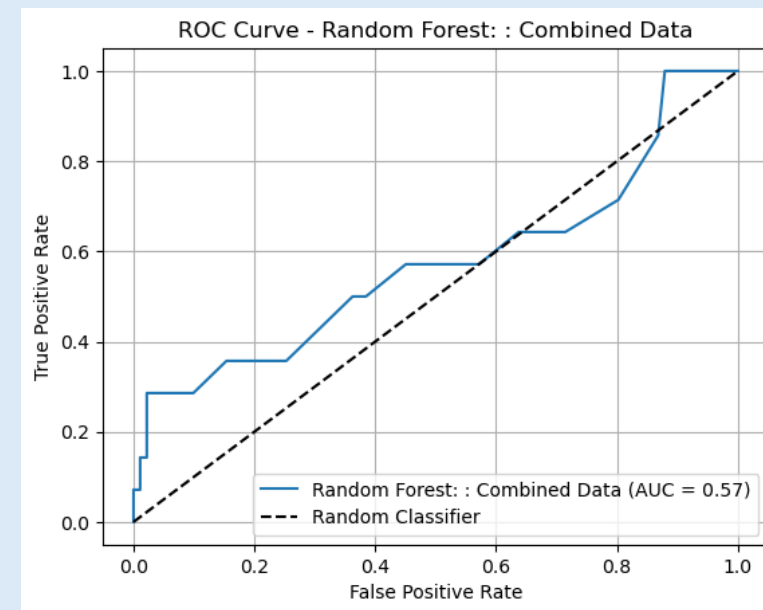
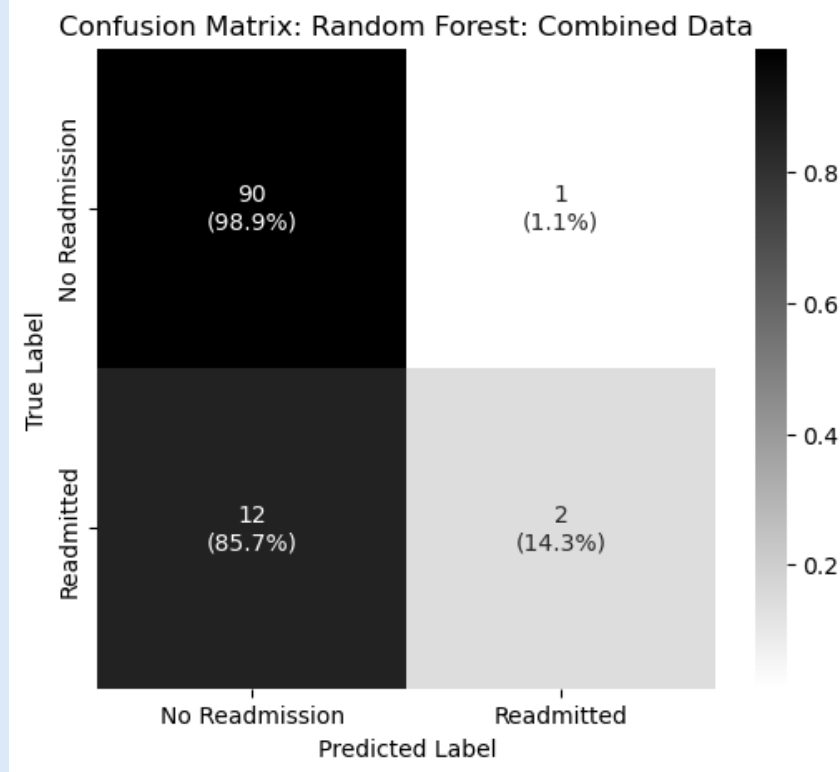
```
# Load both DataFrames
df_structured = pd.read_pickle("structured_data.pkl")
df_bert_embeddings =
pd.read_pickle("bert_embeddings.pkl")

# Merge on hadm_id
final_df = df_structured.merge(df_bert_embeddings,
on='hadm_id', how='inner')

print("Final merged shape:", final_df.shape)
print(final_df.head())
```

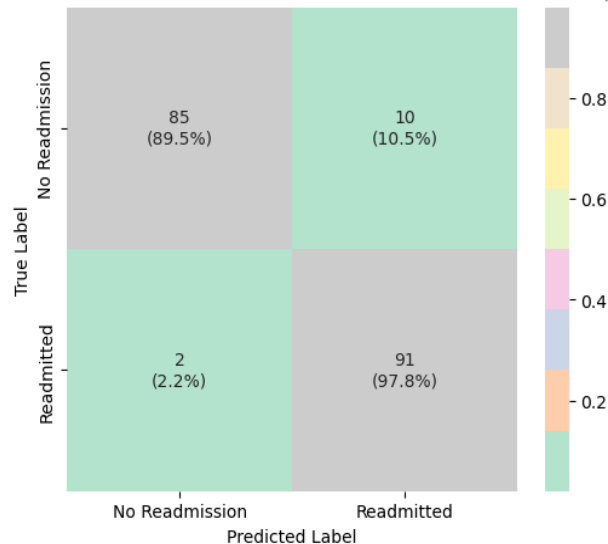


Results: Combined Before SMOTE

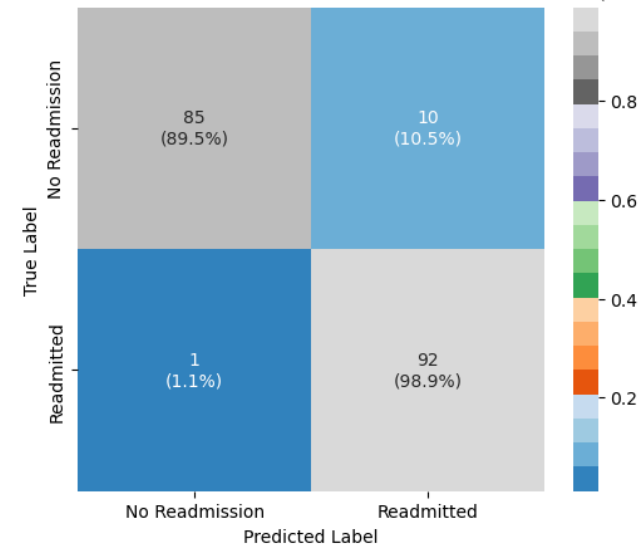


Results: Combined Post SMOTE

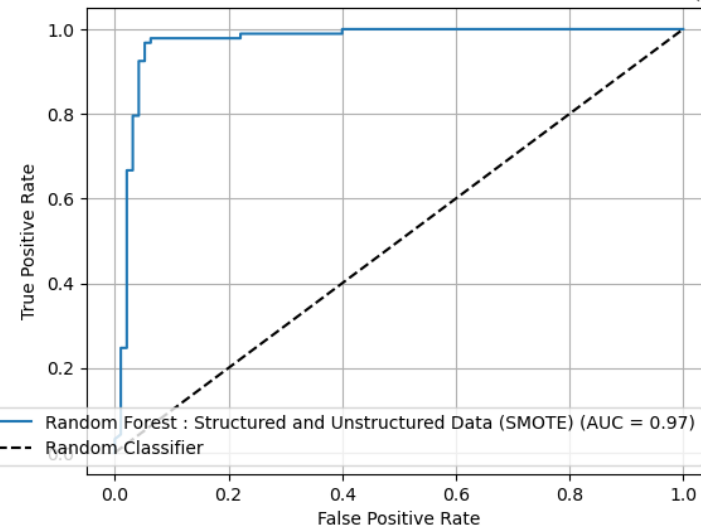
Confusion Matrix: Random Forest : Structured and Unstructured Data (SMOTE)



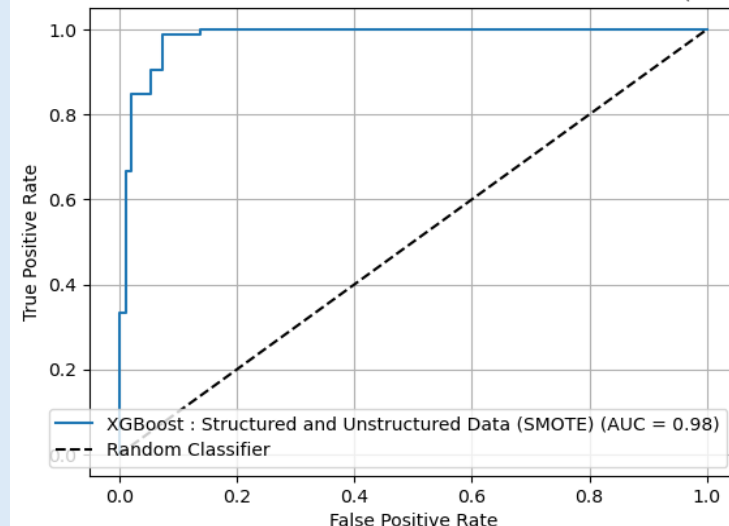
Confusion Matrix: XGBoost : Structured and Unstructured Data (SMOTE)



ROC Curve - Random Forest : Structured and Unstructured Data (SMOTE)

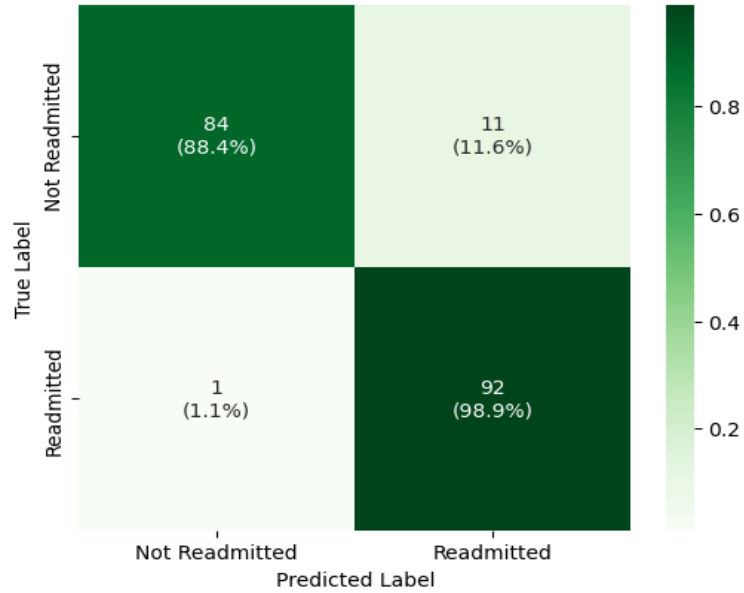


ROC Curve - XGBoost : Structured and Unstructured Data (SMOTE)

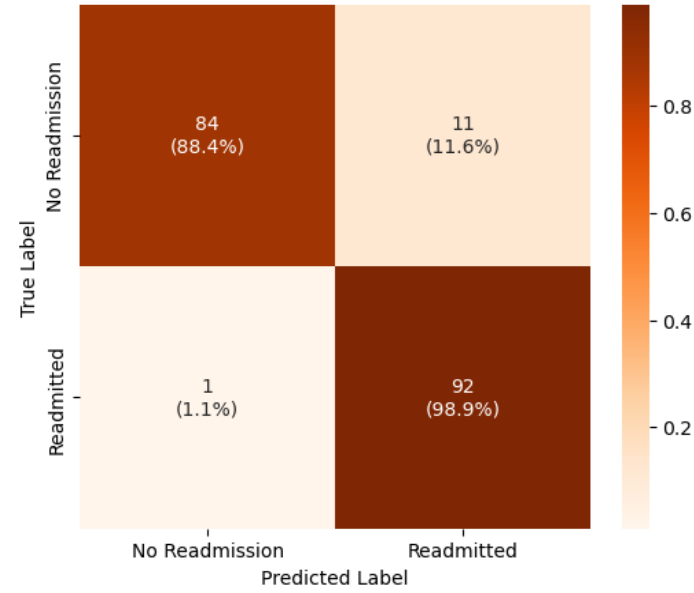


Results: Combined

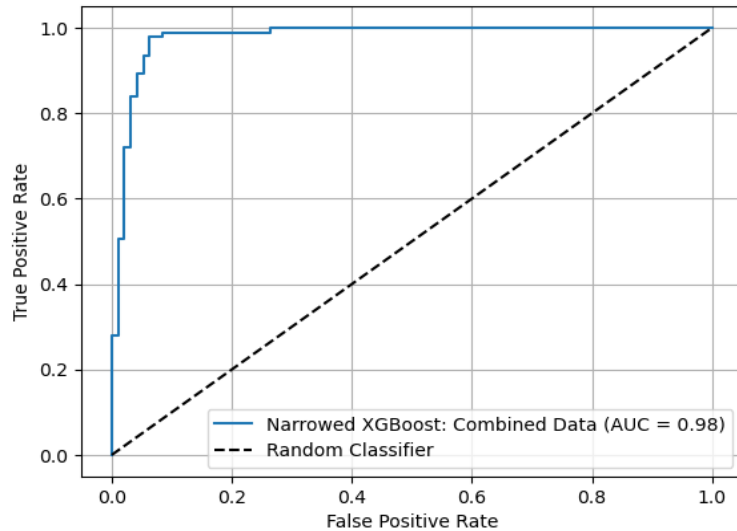
Confusion Matrix: Narrowed XGBoost Combined Data



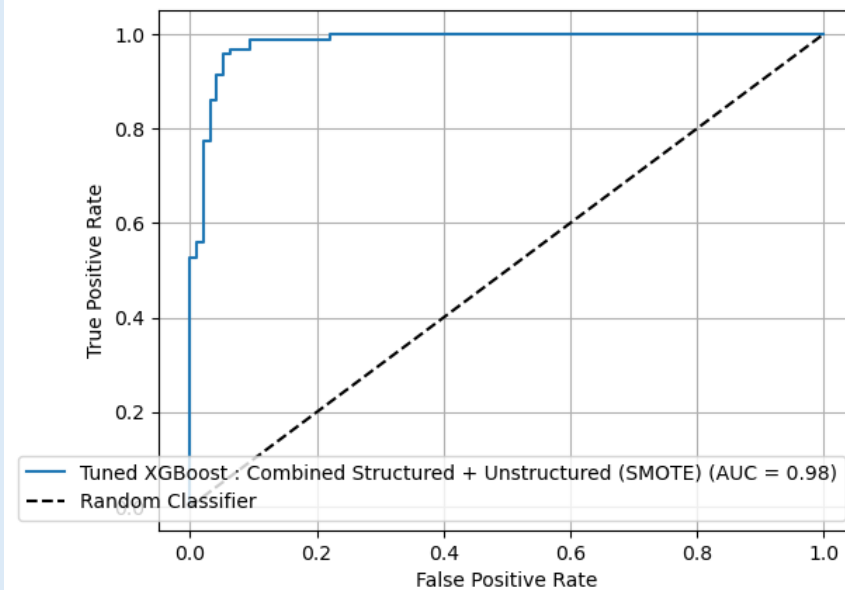
Confusion Matrix: Tuned XGBoost : Combined Structured + Unstructured (SMOTE)



ROC Curve - Narrowed XGBoost: Combined Data

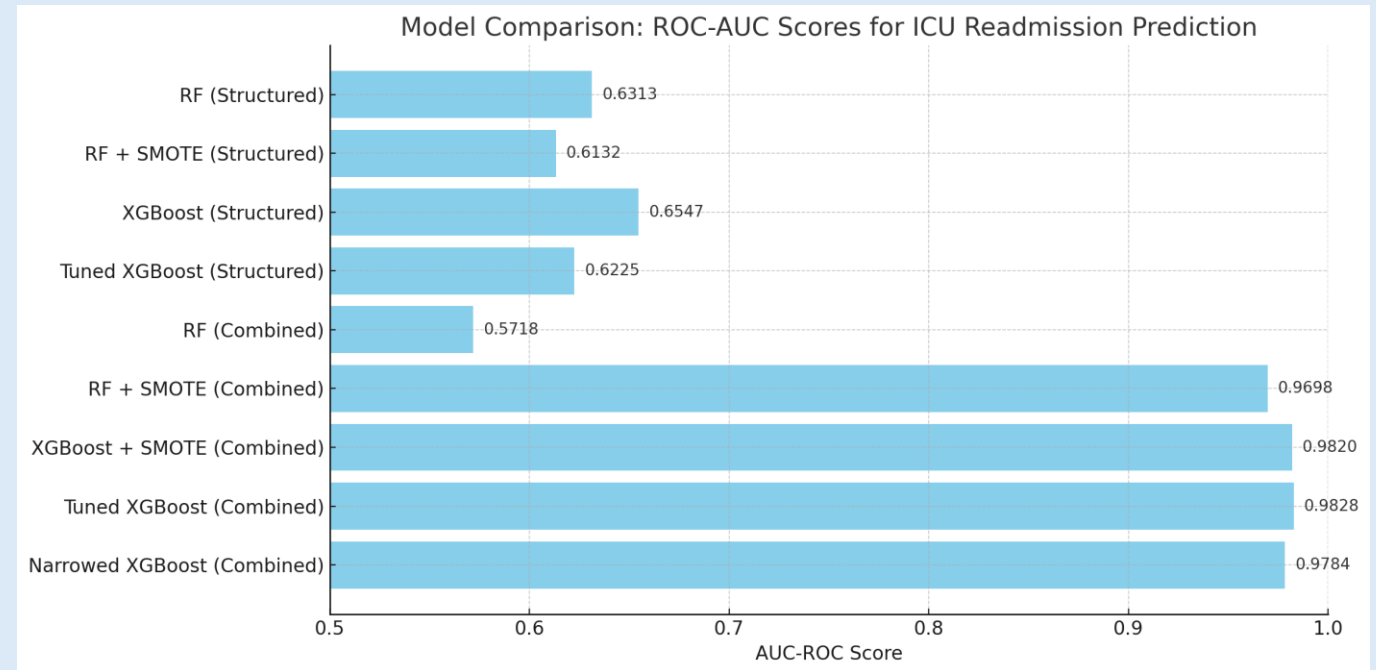


ROC Curve - Tuned XGBoost : Combined Structured + Unstructured (SMOTE)



Conclusion

- Merging structured data with ClinicalBERT-based unstructured text greatly enhances prediction.
- Discharge summaries capture nuanced clinical risk factors missed in vitals/labs.
- SMOTE addressed class imbalance, improving minority class recall.
- Best performance: Narrowed XGBoost on combined data
 - AUROC = 0.98



Conclusion

Limitations

- Small sample size in final test evaluations may limit generalizability.
- ClinicalBERT embeddings were only taken from the first 2500 notes (due to compute constraints).
- SMOTE generates synthetic data, which may not fully capture real patient complexity.
- Lack of external validation on non-MIMIC datasets.

Future Work

- Scale ClinicalBERT embedding to full dataset for stronger representation.
- Explore additional NLP models like BioBERT or Longformer for longer notes.
- Incorporate time-series vitals and medications to enrich structured features.
- Test models on real-time or external hospital data to assess generalizability.
- Apply SHAP or LIME for explainability and model transparency in clinical settings.



References

1. Alsentzer, E., Murphy, J. R., Boag, W., Weng, W.-H., Jin, D., Naumann, T., & McDermott, M. B. A. (2019). Publicly available clinical BERT embeddings [Preprint]. arXiv. <https://arxiv.org/abs/1904.03323>
2. Chiu, C.-C., Wu, C.-M., Chien, T.-N., Kao, L.-J., & Li, C. (2024). Predicting ICU readmission from electronic health records via BERTopic with long short-term memory network approach. *Journal of Clinical Medicine*, 13(5503). <https://doi.org/10.3390/jcm13185503>
3. González-Nóvoa, J. A., Campanioni, S., Busto, L., Fariña, J., Rodríguez-Andina, J. J., Vila, D., Íñiguez, A., & Veiga, C. (2023). Improving intensive care unit early readmission prediction using optimized and explainable machine learning. *International Journal of Environmental Research and Public Health*, 20(3455). <https://doi.org/10.3390/ijerph20043455>
4. Johnson, A. E. W., Pollard, T. J., Shen, L., Li-wei Lehman, L., Feng, M., Ghassemi, M., Moody, B., Szolovits, P., Celi, L. A., & Mark, R. G. (2023). MIMIC-IV (version 2.2). PhysioNet. <https://physionet.org/content/mimiciv/2.2/>
<https://doi.org/10.13026/a3wn-hq05>
5. Johnson, A. E. W., Pollard, T. J., & Mark, R. G. (2023). MIMIC-IV-Note (version 2.2). PhysioNet. <https://physionet.org/content/mimic-iv-note/2.2/>
<https://doi.org/10.13026/9e4v-qw40>
6. Lu, Q., Nguyen, T. H., & Dou, D. (2021). Predicting patient readmission risk from medical text via knowledge graph enhanced multiview graph convolution. *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*, 1989–1994. <https://doi.org/10.1145/3404835.3463062>
7. Orangi-Fard, N., Akhbardeh, A., & Sagreiya, H. (2022). Predictive model for ICU readmission based on discharge summaries using machine learning and natural language processing. *Informatics*, 9(1), 10. <https://doi.org/10.3390/informatics9010010>
8. Rojas, J. C., Carey, K. A., Edelson, D. P., Venable, L. R., Howell, M. D., & Churpek, M. M. (2018). Predicting intensive care unit readmission with machine learning using electronic health record data. *Annals of the American Thoracic Society*, 15(7), 846–853. <https://doi.org/10.1513/AnnalsATS.201710-787OC>
9. Sheetrit, E., Brief, M., & Elisha, O. (2023). Predicting unplanned readmissions in the intensive care unit: a multimodality valuation. *Scientific reports*, 13(1), 15426. <https://doi.org/10.1038/s41598-023-42372-y>

